

1. Algorithm

Main Program Algorithm

1. Start.
2. Declare `choice` as an integer.
3. Display the menu:
 1. Decreasing right-angled triangle
 2. Decreasing natural-number pattern
 3. Decreasing repeated-number pattern
 4. Reverse-number triangle
 5. Exit
4. Read `choice`.
5. Use a `switch` statement:
 - If `choice` = 1, call `decreasingTriangle()`.
 - If `choice` = 2, call `decreasingNatural()`.
 - If `choice` = 3, call `decreasingRepeated()`.
 - If `choice` = 4, call `reverseTriangle()`.
 - If `choice` = 5, exit the program.
 - Otherwise, display "Invalid choice!".
6. Repeat the menu until the user selects 5.
7. Stop.

Activity 1: Decreasing Right-Angled Triangle

1. Start the function.
 2. Input the number of rows `n`.
 3. If `n` <= 0, display an error message and return.
 4. Set `i` = `n`.
 5. Repeat while `i` >= 1:
 - Set `j` = 1.
 - Print * while `j` <= `i`.
 - Move to the next line.
 - Decrease `i` by 1.
 6. Return to the main program.
- For `n` = 4:

```
* * * *
* * *
* *
*
```

Activity 2: Decreasing Natural-Number Pattern

1. Start the function.

2. Input the number of rows n .
3. If $n \leq 0$, display an error message and return.
4. Set $i = n$.
5. Repeat while $i \geq 1$:
 - Set $j = 1$.
 - Print values from 1 through i .
 - Move to the next line.
 - Decrease i by 1.
6. Return to the main program.

For $n = 4$:

```
1 2 3 4
1 2 3
1 2
1
```

Activity 3: Decreasing Repeated-Number Pattern

1. Start the function.
2. Input the number of rows n .
3. If $n \leq 0$, display an error message and return.
4. Set $i = n$.
5. Repeat while $i \geq 1$:
 - Set $j = 1$.
 - Print the value i , i times.
 - Move to the next line.
 - Decrease i by 1.
6. Return to the main program.

For $n = 4$:

```
4 4 4 4
3 3 3
2 2
1
```

Activity 4: Reverse-Number Triangle

1. Start the function.
2. Input the number of rows n .
3. If $n \leq 0$, display an error message and return.
4. Set $i = n$.
5. Repeat while $i \geq 1$:
 - Set $j = i$.

- Print numbers from `i` down to 1.
- Move to the next line.
- Decrease `i` by 1.

6. Return to the main program.

For `n = 4`:

```
4 3 2 1
3 2 1
2 1
1
```

2. Test-Case Table

Test Case	Menu Choice	Input n	Expected Output	Result
1	1	4	4, 3, 2, 1 asterisks in successive rows	Pass
2	2	4	1 2 3 4, 1 2 3, 1 2, 1	Pass
3	3	4	4 4 4 4, 3 3 3, 2 2, 1	Pass
4	4	4	4 4 3 2 1, 3 2 1, 2 1, 1	Pass
5	1	1	*	Pass
6	2	1		1 Pass
7	3	1		1 Pass
8	4	1		1 Pass
9	1	0	Number of rows must be positive.	Pass
10	3	-2	Number of rows must be positive.	Pass
11	7	—	Invalid choice!	Pass
12	5	—	Program terminates	Pass

The cases with `n = 1` test the **boundary condition**, while `n = 0`, negative `n`, and an invalid menu choice test **invalid inputs**.

3. C Program

```
#include <stdio.h>
```

```
/* Function declarations */
void decreasingTriangle(void);
```

```

void decreasingNatural(void);
void decreasingRepeated(void);
void reverseTriangle(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n===== DECREASING PATTERNS
=====\\n");
        printf("1. Decreasing right-angled triangle\\n");
        printf("2. Decreasing natural-number pattern\\n");
        printf("3. Decreasing repeated-number pattern\\n");
        printf("4. Reverse-number triangle\\n");
        printf("5. Exit\\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                decreasingTriangle();
                break;

            case 2:
                decreasingNatural();
                break;

            case 3:
                decreasingRepeated();
                break;

            case 4:
                reverseTriangle();
                break;

            case 5:
                printf("Exiting the program...\\n");
                break;

            default:

```

```

                printf("Invalid choice! Please enter 1 to
5.\n");
            }

        } while (choice != 5);

    return 0;
}

```

```

/* 1. Decreasing right-angled triangle */

```

```

void decreasingTriangle(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

```

```

/* 2. Decreasing natural-number pattern */

```

```

void decreasingNatural(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {

```

```

        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}

```

/* 3. Decreasing repeated-number pattern */

```

void decreasingRepeated(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

```

/* 4. Reverse-number triangle */

```

void reverseTriangle(void)
{
    int n, i, j;

```

```

printf("Enter the number of rows: ");
scanf("%d", &n);

if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = n; i >= 1; i--)
{
    for (j = i; j >= 1; j--)
    {
        printf("%d ", j);
    }
    printf("\n");
}
}

```

5.. Sample Execution

Test 1 — Decreasing Right-Angled Triangle

```

Last login: Thu Sep 17 08:49:01 on ttys000
[bhavya@Bhavyas-MacBook-Air ~ % nano day25.cpp
[bhavya@Bhavyas-MacBook-Air ~ % clang day25.cpp -o day25
[bhavya@Bhavyas-MacBook-Air ~ % ./day25

```

```

===== DECREASING PATTERNS =====

```

1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit

Enter your choice: 1

Enter the number of rows: 4

```

* * * *
* * *
* *
*

```

```

===== DECREASING PATTERNS =====

```

1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit

Test 2 — Decreasing Natural-Number Pattern

```
Enter your choice: 2
Enter the number of rows: 4
1 2 3 4
1 2 3
1 2
1

===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 2
```

Test 3 — Decreasing Repeated-Number Pattern

```
Enter your choice: 3
Enter the number of rows: 4
4 4 4 4
3 3 3
2 2
1

===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
```

Test 4 — Reverse-Number Triangle

```

5. Exit
Enter your choice: 4
Enter the number of rows: 4
4 3 2 1
3 2 1
2 1
1

===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit

```

Invalid Input Test

```

Enter your choice: 1
Enter the number of rows: 0
Number of rows must be positive.

===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit

```

Exit

```

Enter your choice: 8
Invalid choice! Please enter 1 to 5.

===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 5
Exiting the program...
bhavya@Bhavyas-MacBook-Air ~ %

```

Conclusion: All four decreasing patterns are generated using **nested loops without arrays**, each operation is implemented using a separate **parameterless void function**, and the required operation is selected using a **switch statement**.